

Machine Learning (CAI-262)

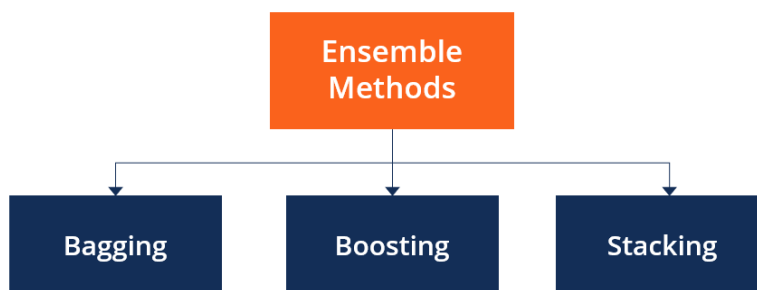
Objectives:

- Ensemble Methods
 - Random Forest
 - XGBoost
-

Ensemble Methods

Ensemble methods are techniques which improve the accuracy of results in models by combining multiple models instead of using a single model. The combined models increase the accuracy of the results significantly. This has boosted the popularity of ensemble methods in machine learning.

The most popular ensemble methods are boosting, bagging, and stacking.



Categories of Ensemble Methods

Ensemble methods fall into two broad categories, i.e., sequential ensemble techniques and parallel ensemble techniques.

Sequential ensemble techniques generate base learners in a sequence, e.g., Adaptive Boosting (AdaBoost). The sequential generation of base learners promotes the dependence between the base learners. The performance of the model is then improved by assigning higher weights to previously misrepresented learners.

Parallel ensemble techniques, in this base learners are generated in a parallel format, e.g. random forest. Parallel methods utilize the parallel generation of base learners to encourage independence between the base learners. The independence of base learners significantly reduces the error due to the application of averages.

Main Types of Ensemble Methods

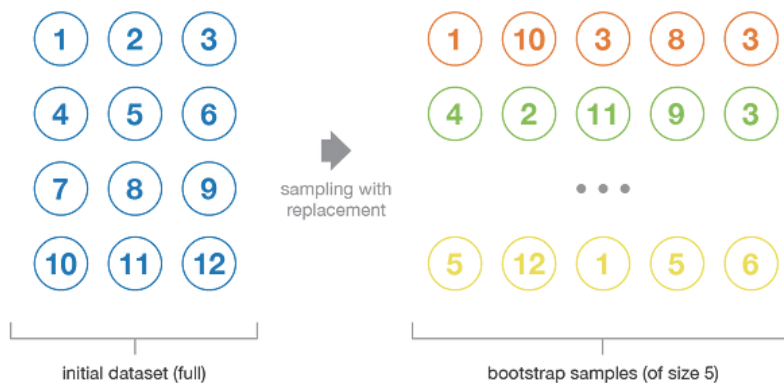
1. Bagging

Bagging, the short form for **bootstrap aggregating**, is mainly applied in classification and regression. It increases the accuracy of models through decision trees, which reduces variance to a large extent. The reduction of variance increases accuracy, eliminating overfitting, which is a challenge to many predictive models.

Bagging Involves having each model in the ensemble vote with equal weight. It trains each model in the ensemble using a randomly drawn subset of the training set. Its example is **Random Forest algorithm**

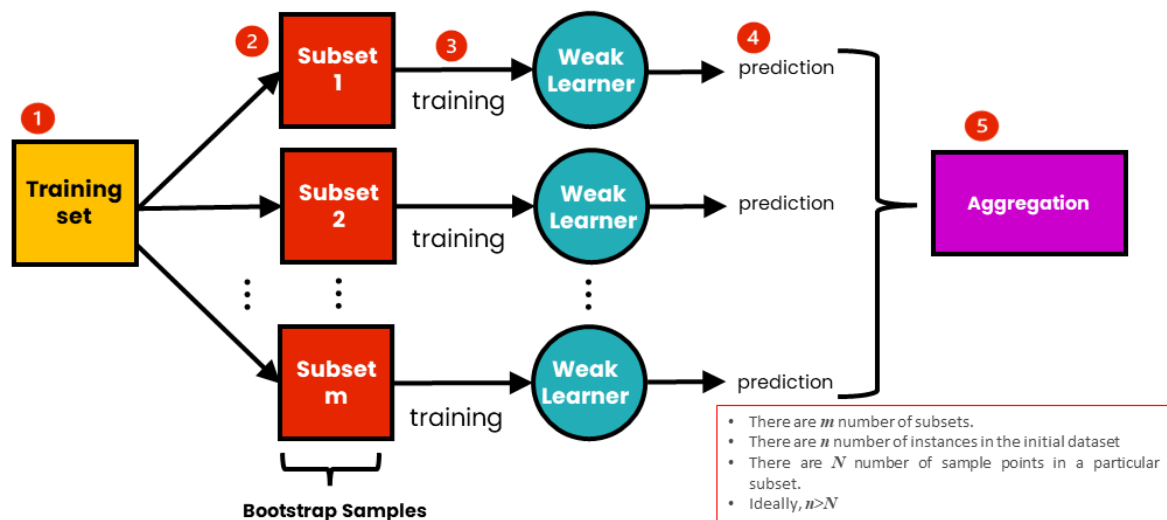
Bagging is classified into two types, i.e., bootstrapping and aggregation.

Bootstrapping is a sampling technique where samples are derived from the whole population (set) using the replacement procedure. The sampling with replacement method helps make the selection procedure randomized.



Aggregation in bagging is done to incorporate all possible outcomes of the prediction and randomize the outcome. Without aggregation, predictions will not be accurate because all outcomes are not put into consideration. Therefore, the aggregation is based on the probability bootstrapping procedures or on the basis of all outcomes of the predictive models.

The Process of Bagging (Bootstrap Aggregation)



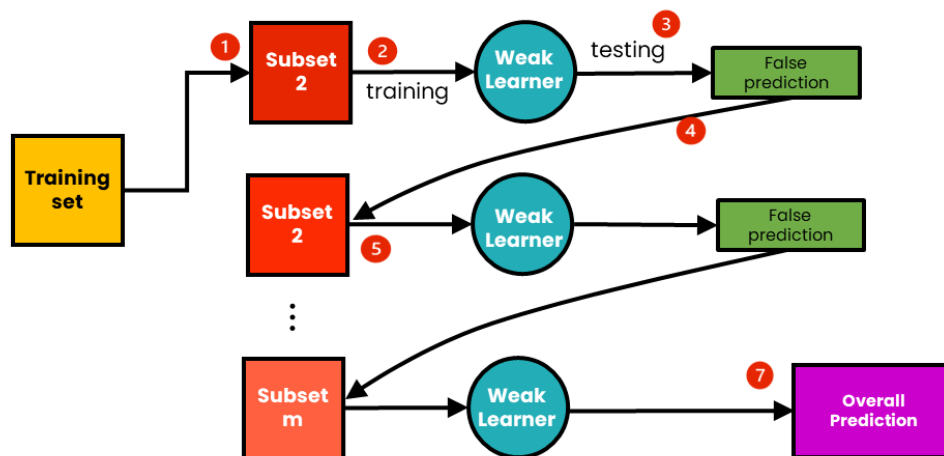
Bagging is advantageous since weak base learners are combined to form a single strong learner that is more stable than single learners. It also eliminates any variance, thereby reducing the overfitting of models. One limitation of bagging is that it is computationally expensive. Thus, it can lead to more bias in models when the proper procedure of bagging is ignored.

2. Boosting

Boosting involves incrementally building an ensemble by training each new model instance to emphasize the training instances that previous models mis-classified. Boosting is an ensemble technique that learns from previous predictor mistakes to make better predictions in the future. The technique combines several weak base learners to form one strong learner, thus significantly improving the predictability of models. Boosting works by arranging weak learners in a sequence, such that weak learners learn from the next learner in the sequence to create better predictive models.

Examples are Adaptive Boosting (AdaBoost), and XGBoost (Extreme Gradient Boosting).

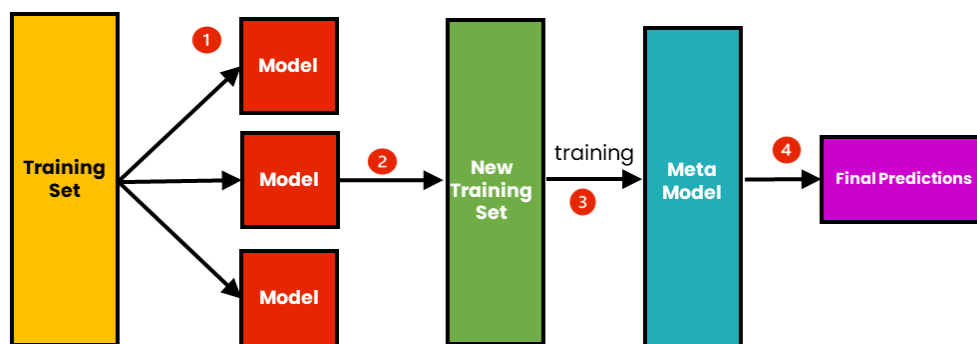
The Process of Boosting



3. Stacking

Stacking, another ensemble method, is often referred to as *stacked generalization*. It builds a model that predicts the output based on the prediction of individual models. This technique works by allowing a training algorithm to ensemble several other similar learning algorithm predictions.

The Process of Stacking



Bias and Variance:

Bias is one type of error that occurs due to wrong assumptions about data such as assuming data is linear when in reality, data follows a complex function. On the other hand, variance gets introduced with high sensitivity to variations in training data. This also is one type of error since we want to make our model robust against noise.

There are two types of error in machine learning. Reducible error and Irreducible error. Bias and Variance come under reducible error.

Bias

These differences between actual or expected values and the predicted values are known as error or bias error or error due to bias.

- **Low Bias:** Low bias value means fewer assumptions are taken to build the target function. In this case, the model will closely match the training dataset.
- **High Bias:** High bias value means more assumptions are taken to build the target function. In this case, the model will not match the training dataset closely.

Variance

Variance is the measure of spread in data from its mean position. In machine learning variance is the amount by which the performance of a predictive model changes when it is trained on different subsets of the training data. More specifically, variance is the variability of the model that how much it is sensitive to another subset of the training dataset. i.e. how much it can adjust on the new subset of the training dataset.

Variance errors are either low or high-variance errors.

- **Low variance:** Low variance means that the model is less sensitive to changes in the training data and can produce consistent estimates of the target function with different subsets of data from the same distribution. This is the case of underfitting when the model fails to generalize on both training and test data.
- **High variance:** High variance means that the model is very sensitive to changes in the training data and can result in significant changes in the estimate of the target function when trained on different subsets of data from the same distribution. This is the case of overfitting when the model performs well on the training data but poorly on new, unseen test data. It fits the training data too closely that it fails on the new training dataset.

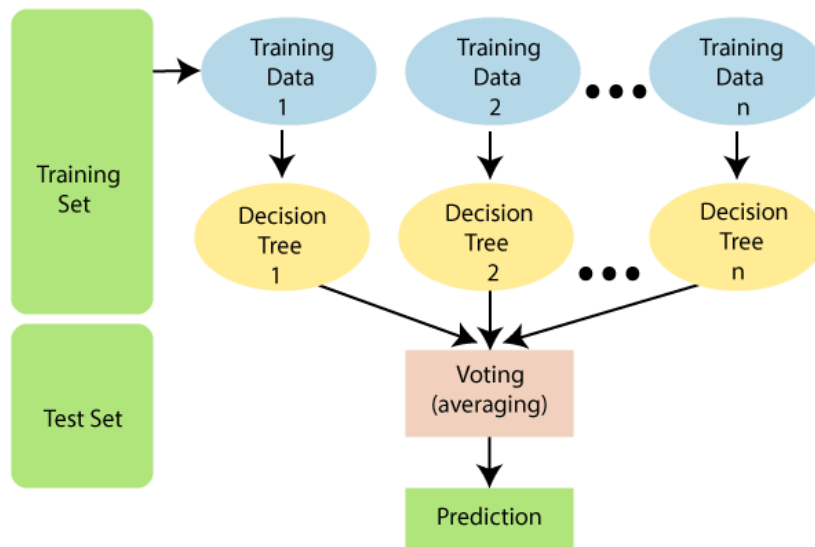
Bagging aims to produce a model with lower variance than the individual weak models. These weak learners are homogenous, meaning they are of the same type.

Boosting aims to produce a model with a lower bias than that of the individual models. Like in bagging, the weak learners are homogeneous.

Random Forest

Random Forest is a classifier that contains a number of decision trees on various subsets of the given dataset and takes the average to improve the predictive accuracy of that dataset.

Instead of relying on one decision tree, the random forest takes the prediction from each tree and based on the majority votes of predictions, and it predicts the final output. The greater number of trees in the forest leads to higher accuracy and prevents the problem of overfitting.



The Working process of random forest can be explained in the below steps and diagram:

Step-1: Select random K data points from the training set.

Step-2: Build the decision trees associated with the selected data points (Subsets).

Step-3: Choose the number N for decision trees that you want to build.

Step-4: Repeat Step 1 & 2.

Step-5: For new data points, find the predictions of each decision tree, and assign the new data points to the category that wins the majority votes.

Advantages of Random Forest

Random Forest is capable of performing both Classification and Regression tasks.

It is capable of handling large datasets with high dimensionality.

It enhances the accuracy of the model and prevents the overfitting issue.

Disadvantages of Random Forest

Although random forest can be used for both classification and regression tasks, it is not more suitable for Regression tasks.

XGBoost:

As it's an Ensemble technique, we are grouping very shallow decision trees and using them to produce predictions. We build High bias and low-variance trees and combine them additively to reduce bias without altering the variance.

$$\text{Model error} = \text{bias}^2 + \text{variance} + \epsilon$$

This is the error with which every ML model suffers, epsilon is an irreducible error. The way we are building these shallow Decision trees is what makes the difference. Basically, like every other decision tree, we start with the root node and try to create a group of the data by splitting to Maximize Gain, that's how we create every level of the decision tree.

$$\text{Gain} = (SS_l + SS_r) - SS_{\text{root}}$$

$$SS = \frac{(\text{Sum of residuals})^2}{\text{No. of residual} + \lambda}$$

$$Ov = \frac{\text{Sum of residuals}}{\text{No. of residual} + \lambda}$$

$$\text{Prediction} = \text{Base score} + \eta \cdot (T_1 + T_2 \dots T_n)$$

SS_l : Similarity score of the left node

SS_r : Similarity score of the right node

SS_{root} : Similarity score of the root node

λ : Regularization parameter

Ov : Output value at leaf node

η : Learning rate

Base score : Any value of choice

T_i : Ov from the i th tree

residual : Actual value Predicted value

The gradient boosting ensemble technique consists of three simple steps:

1. An initial model F_0 is defined to predict the target variable y . This model will be associated with a residual $(y - F_0)$
2. A new model h_1 is fit to the residuals from the previous step.
3. Now, F_0 and h_1 are combined to give F_1 , the boosted version of F_0 . The mean squared error from F_1 will be lower than that from F_0 :

$$F_1(x) <- F_0(x) + h_1(x)$$

4. To improve the performance of F_1 , we could model after the residuals of F_1 and create a new model F_2 :

$$F_2(x) <- F_1(x) + h_2(x)$$

5. This can be done for 'm' iterations, until residuals have been minimized as much as possible:

$$F_m(x) <- F_{m-1}(x) + h_m(x)$$

Here, the additive learners do not disturb the functions created in the previous steps. Instead, they impart information of their own to bring down the errors.

XGBoost uses the following parameters and methods to optimize the algorithm and provide better results and performance:

- **Regularization**—A Regularization parameter (**lambda**) is used while calculating the similarity scores to reduce the sensitivity to individual data and avoid overfitting.
- **Pruning**—A Tree Complexity Parameter (**gamma**) is selected to compare the gains. The branch where the gain is smaller than the gamma value is removed. This prevents overfitting by trimming unnecessary branches and reducing the depth of the trees.

- Weighted quantile sketch—Instead of testing every possible value as the threshold for splitting the data, only weighted quantiles are used. The selection of quantiles is done using a sketch algorithm, which estimates a distribution on multiple systems over a network.